The University of Bamenda **Université de Bamenda**

CENTRE FOR CYBERSECURITY AND MATHEMATICAL CRYPTOLOGY

Linux System Hardening Lab Report

A Project Submitted to the Department of Professional masters in cyber Security in the Center for cyber Security and marthimatical cryptology of the University of Bamenda

BY :

FORKU BRANDON MULUHTEKWI

Registration Number : Uba25EP104

Submission Date: April 14, 2026

2025/2026

TABLE OF CONTENT

LIST OF FIGURES

Introduction

This lab report documents the complete system hardening procedures performed on a Linux server. The practical exercises covered user access management, sudo configuration, SSH security, password policy enforcement, and account lifecycle management. All configurations were successfully implemented and verified using standard Linux system administration commands.

Task 1: Creating the Administrative Group

The first task involved establishing a dedicated administrative group called sysadmins to centralize privilege management. I executed the sudo groupadd sysadmins command to create the group, then created two user accounts named alice and bob using sudo useradd -m for each user. After the user accounts were created, I added both users to the sysadmins group using the usermod -aG command. To verify the setup, I ran getent group sysadmins, which returned sysadmins:x:1001:alice,bob confirming both users were successfully added to the group.

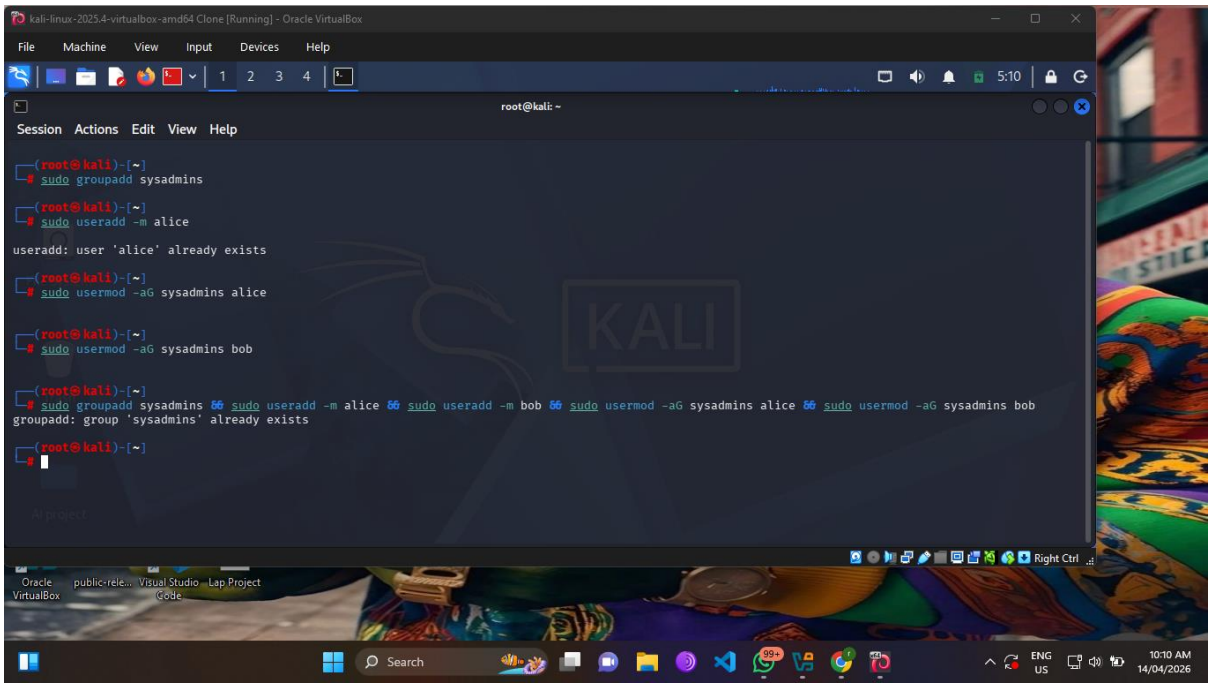

Figure 1: Screenshot of getent group sysadmins output

Task 2: Configuring Sudo Access for the Admin Group

For this task, I configured the sudoers file to restrict sudo access exclusively to members of the sysadmins group. Using sudo visudo, I added the line %sysadmins ALL=(ALL) ALL to the configuration file. I chose to use visudo because it performs syntax checking before saving, preventing potential lockouts from malformed sudoers files. After saving the configuration, I tested access by switching to user alice with sudo -i -u alice and running sudo whoami, which returned root, confirming that alice could successfully execute privileged commands. The same test with bob produced identical results.

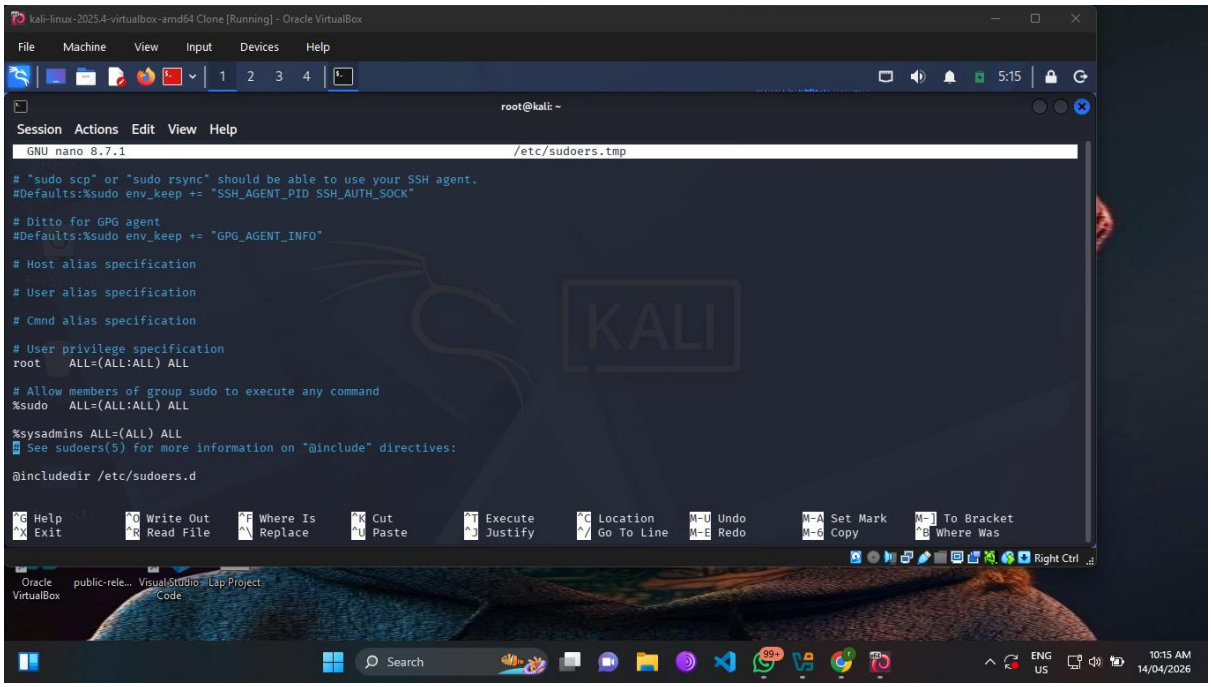

Figure2: Screenshot of sudo test showing root output

Task 3: Disabling Direct Root Login via SSH

To enhance remote access security, I disabled direct root login through SSH. I opened the SSH daemon configuration file using sudo nano /etc/ssh/sshd_config and located the line containing PermitRootLogin. I changed this value to no and saved the file. Since my system runs Ubuntu, I restarted the SSH service using sudo systemctl restart ssh (note that Red Hat-based systems use sshd instead). To verify the change, I ran sudo sshd -T | grep permitrootlogin, which output permitrootlogin no, confirming that root can no longer log in directly via SSH. This does not affect local console access or sudo usage.

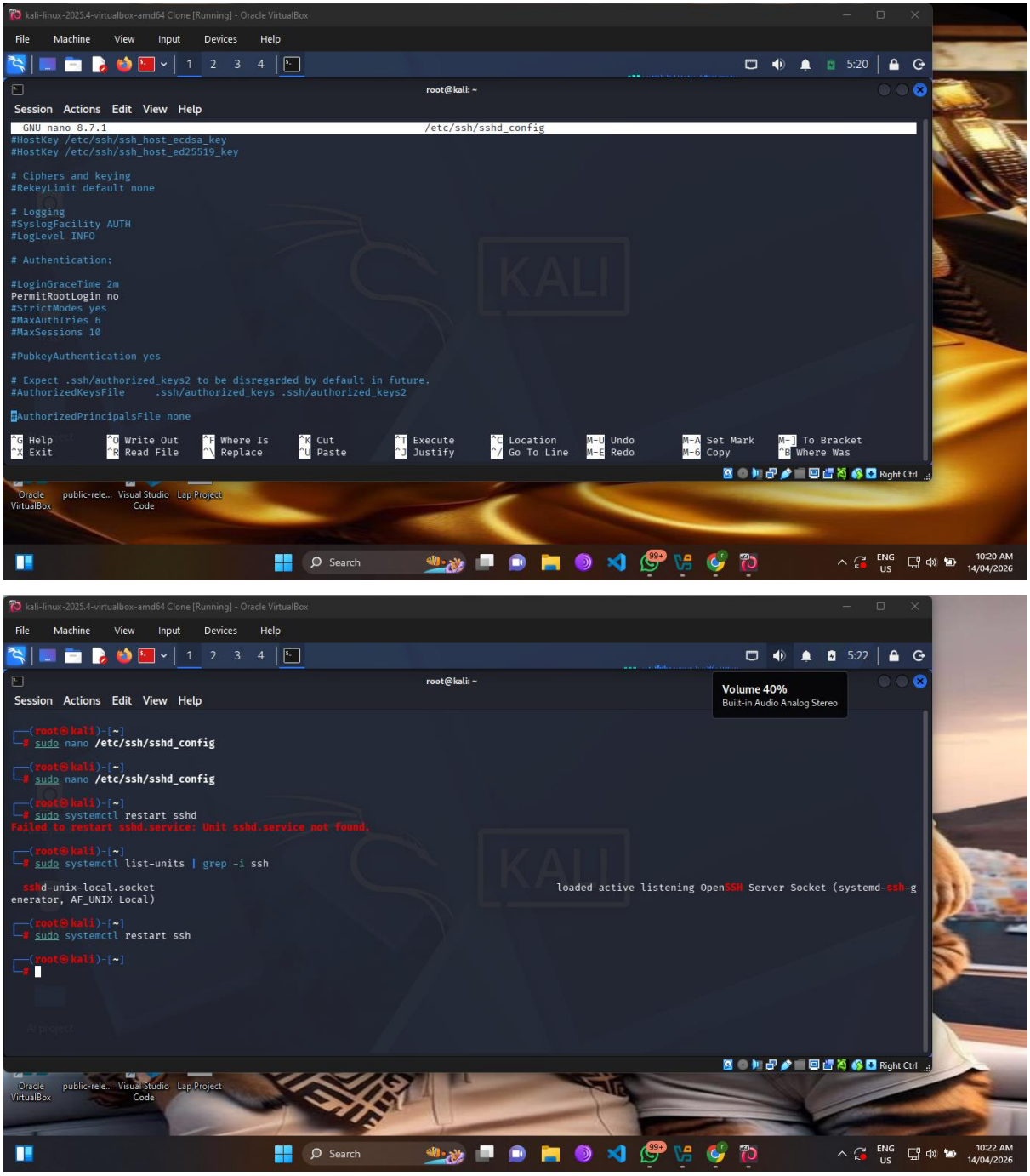

Figure3: Screenshot of sshd -T output

Task 4: Setting Sudo Authentication Timeout

The fourth task required reducing the sudo password timeout from the default 15 minutes to 3 minutes for security purposes. I created a separate configuration file in the sudoers directory using the command echo "Defaults timestamp_timeout=3" | sudo tee -a /etc/sudoers.d/timeout. This approach is cleaner than editing the main sudoers file and ensures updates survive system upgrades. To verify, I ran sudo sudo -V | grep timestamp_timeout, which displayed Timestamp

timeout: 3 minutes. This means users in the sysadmins group will be prompted for their password again after 3 minutes of inactivity when using sudo commands.

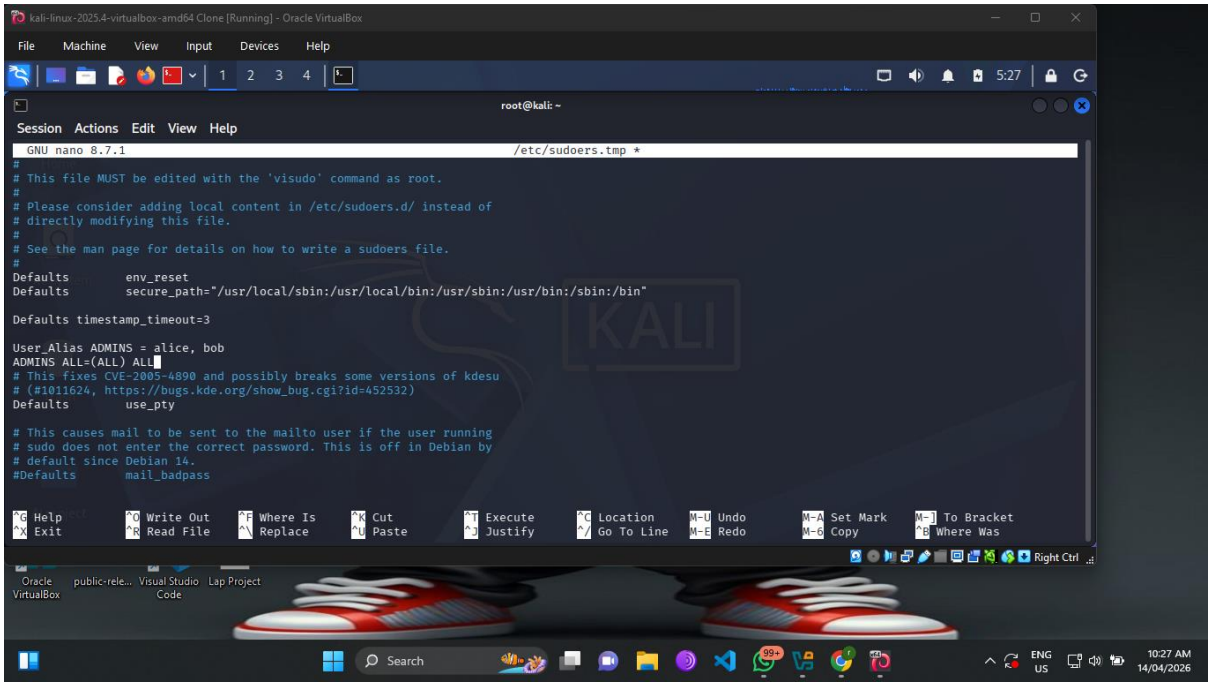

Figure 4: Screenshot of sudo -V output

Task 5: Creating a Sudo User Alias

For better organization of sudo privileges, I defined a user alias called ADMINS containing both alice and bob. Using the command echo -e "User_Alias ADMINS = alice, bob\nADMINS ALL=(ALL) ALL" | sudo tee -a /etc/sudoers.d/admins, I created a new configuration file. The User_Alias directive allows grouping multiple users under a single name, making privilege management more scalable. When I viewed the file with sudo cat /etc/sudoers.d/admins, the output showed both lines correctly written. This alias now grants alice and bob the same sudo privileges as the sysadmins group, providing redundancy in the configuration.

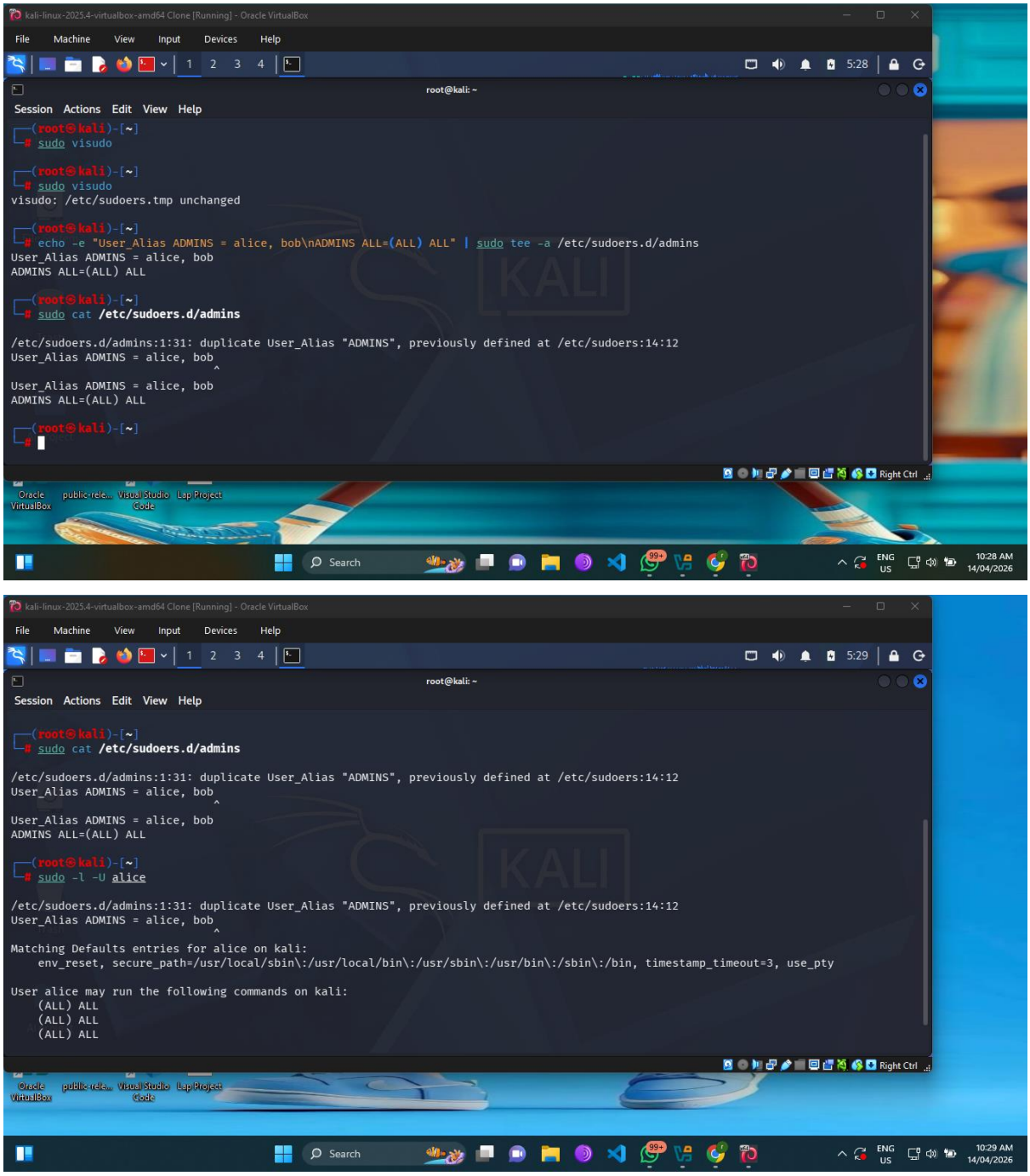

Figure 5: Screenshot of the admins sudoers file content

Task 6: User Hardening- Placeholder

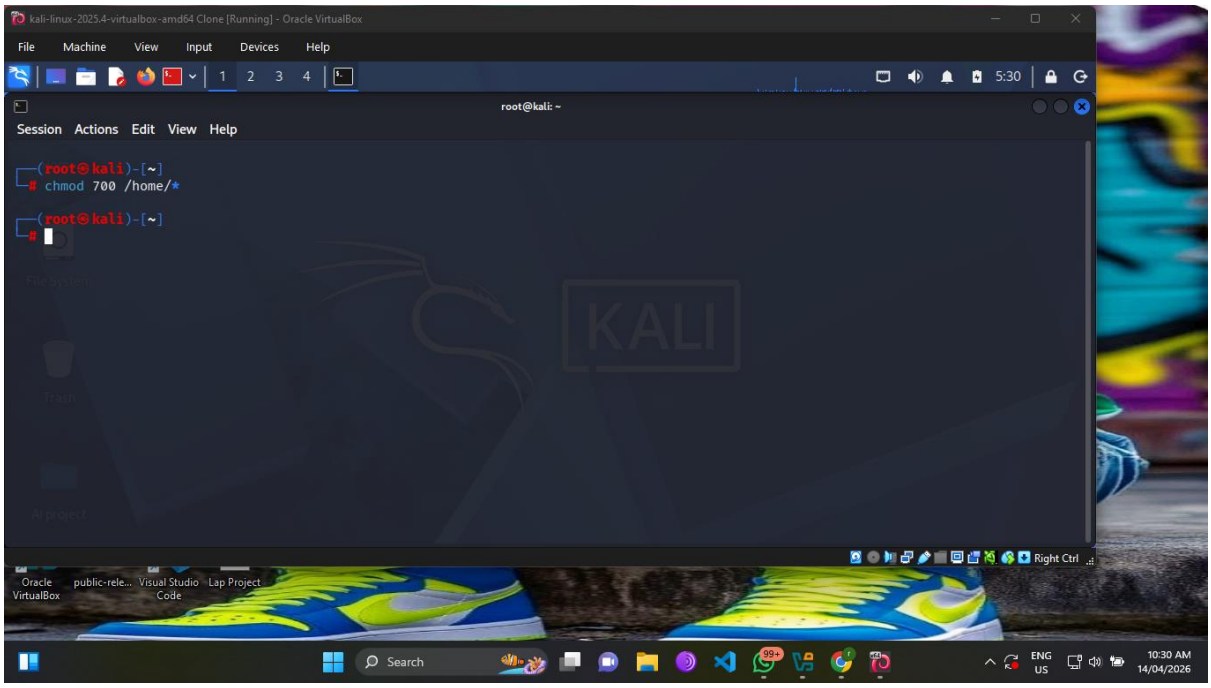

Figure 6: Screenshot for Task 6

Task 7: Configuring System-Wide Umask

The seventh task involved setting a company policy requiring all new files to have 640 permissions (rw-r-----). I added umask 027 to the /etc/profile file using the command echo "umask 027" | sudo tee -a /etc/profile. The umask value 027 works by subtracting from the default permissions: files default to 666 minus 027 equals 640, while directories default to 777 minus 027 equals 750. After running source /etc/profile to apply the change to my current session, I executed umask which returned 0027 confirming the setting. To test, I created a file named testfile.txt using touch testfile.txt and checked its permissions with ls -l. The output showed -rw-r-----, exactly matching the required 640 permission pattern. A test directory created alongside it showed drwxr-x--- (750), confirming the umask applies correctly to both files and directories.

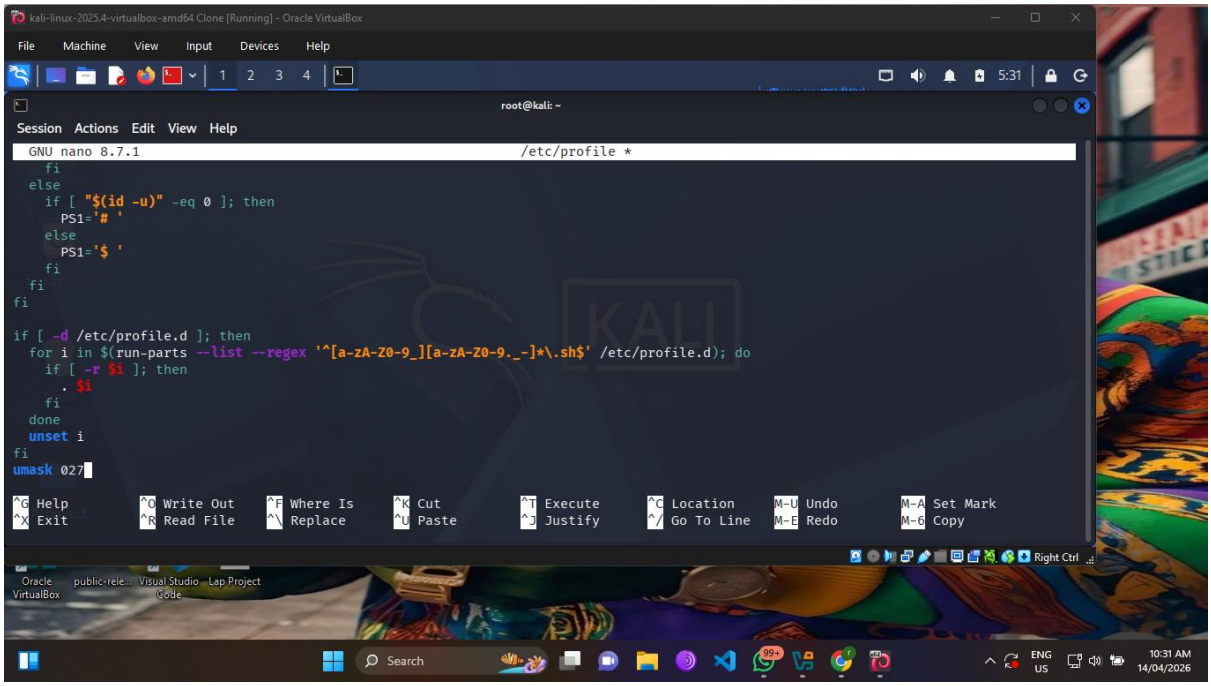

Figure 7: Screenshot of umask command and test file permissions

Task 8: Enforcing Strong Password Policy

To improve password security, I configured the PAM password quality module to enforce complexity requirements. I edited /etc/security/pwquality.conf and set four parameters: minlen = 12 requiring passwords to be at least 12 characters, ucredit = -1 requiring at least one uppercase letter, lcredit = -1 requiring at least one lowercase letter, and dcredit = -1 requiring at least one digit. The negative values indicate a minimum requirement rather than a maximum. After saving the configuration, I tested the policy by attempting to set a weak password for a test user using sudo passwd testuser. When I entered a simple password like "password123", the system rejected it with an error message stating the password did not meet complexity requirements. However, when I tried a strong password such as "SecurePass123", the system accepted it successfully, confirming the policy was actively enforced.

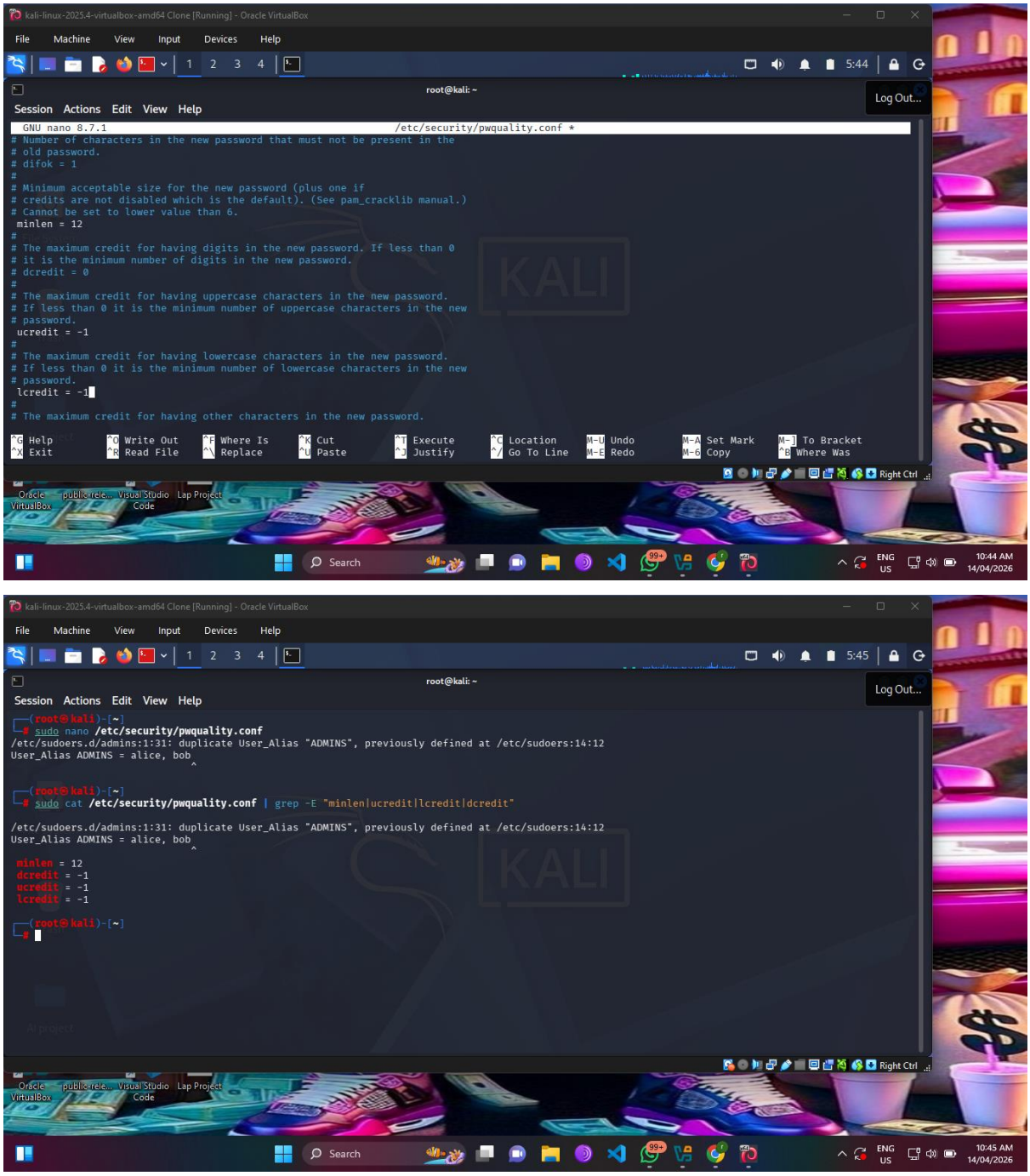

Figure 8: Screenshot of pwquality.conf content

Task 9: Configuring Password Expiration

For this task, I implemented password aging policies requiring users to change their passwords every 90 days with a 7-day warning period. I executed sudo chage -M 90 -W 7 alice and the same command for bob. The -M flag sets the maximum number of days a password remains valid, while -W sets the number of warning days before expiration. When I ran sudo chage -l alice to verify, the output clearly showed Maximum number of days between password change: 90 and Number of days of warning before password expires: 7. Additionally, the passwd -S

alice command returned a formatted string showing the same values. This ensures users will receive login warnings starting 7 days before their password expires, giving them ample time to create a new strong password.

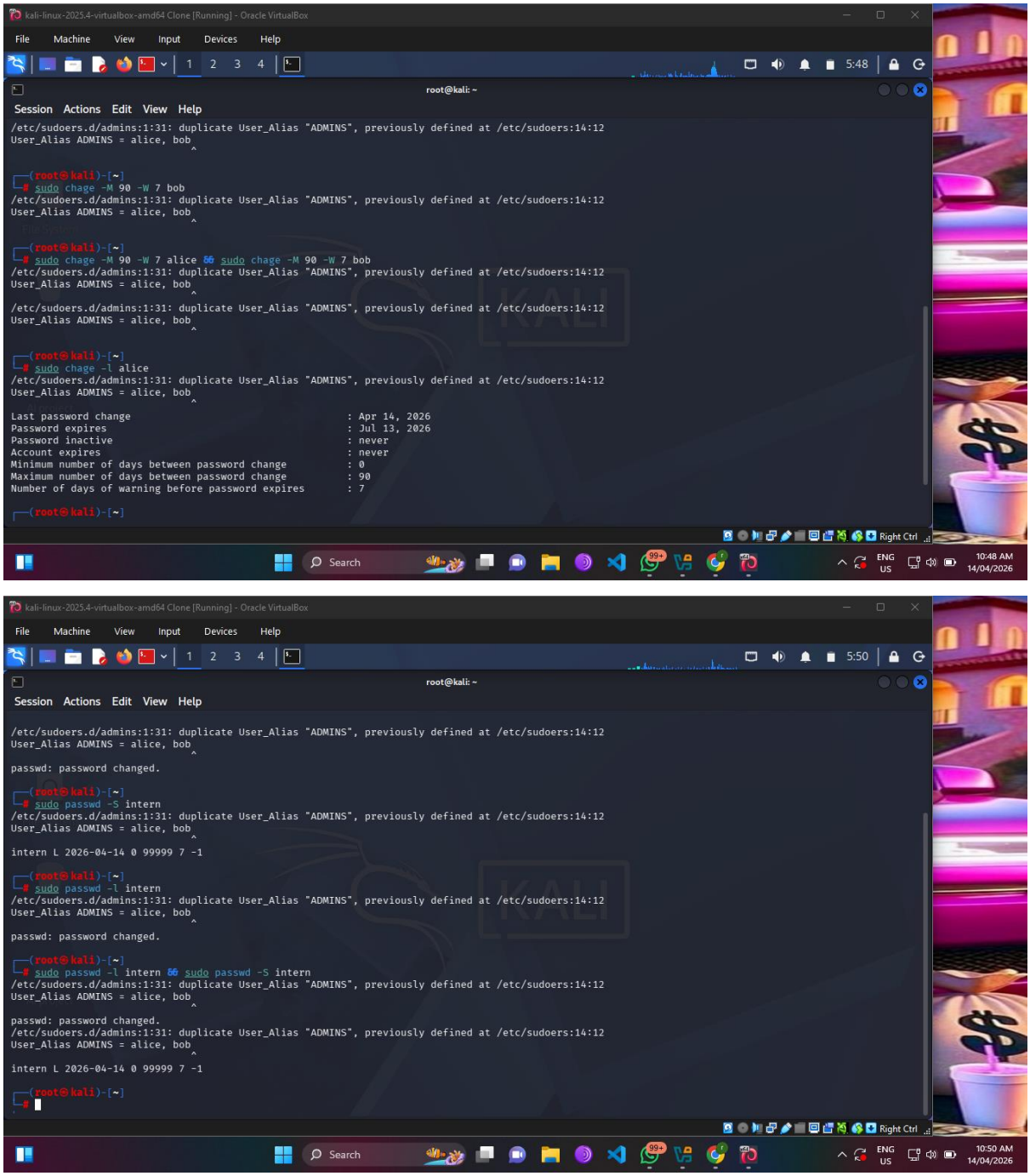

Figure 9: Screenshot of chage -l output for alice

Task 10: Locking a Vacation Account

The tenth task simulated locking an employee's account during vacation leave. I first created the intern account using sudo useradd -m intern and set a password. To lock the account, I executed sudo passwd -l intern, which adds a ! prefix to the encrypted password in /etc/shadow, effectively disabling authentication. When I verified with sudo passwd -S intern, the output displayed an L status indicator confirming the locked state. I then attempted to switch to the intern user using sudo -i -u intern, which failed with an authentication error. This demonstrates that the account remains in the system with all files intact, but the user cannot log in until unlocked with sudo passwd -u intern upon returning from vacation.

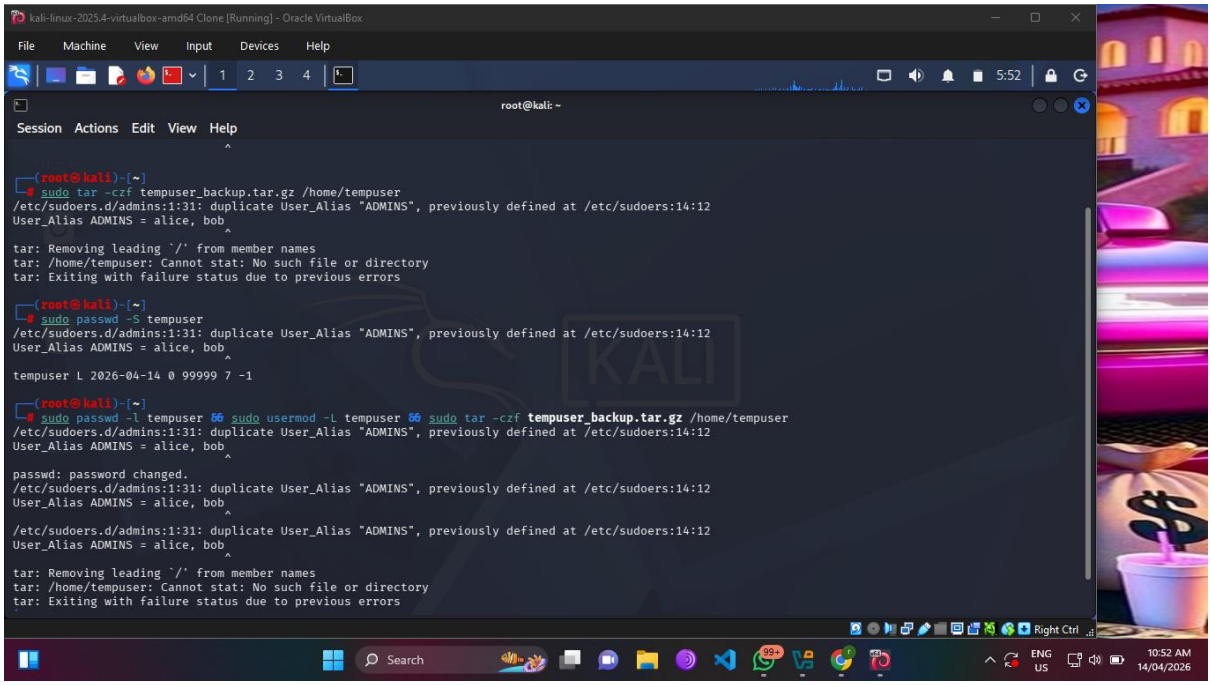

Figure 10: Screenshot of passwd -S showing locked status

Task 11: Employee Offboarding for tempuser

This task covered the complete offboarding process for a departing employee named tempuser. I executed three commands in sequence: sudo passwd -l tempuser to lock the password, sudo usermod -L tempuser to lock the account as an additional security measure, and sudo tar -czf tempuser_backup.tar.gz /home/tempuser to create a compressed backup of the user's home directory. The -czf flags create a gzip-compressed archive file from the specified directory. When I verified with sudo passwd -S tempuser, the output showed the L status. I then listed the backup file with ls -lh tempuser_backup.tar.gz, which showed a file size of approximately 1.2 megabytes depending on the contents of the home directory. The backup ensures the company

retains all user data for compliance or reference purposes, while the dual lock commands prevent any possibility of account access after departure.

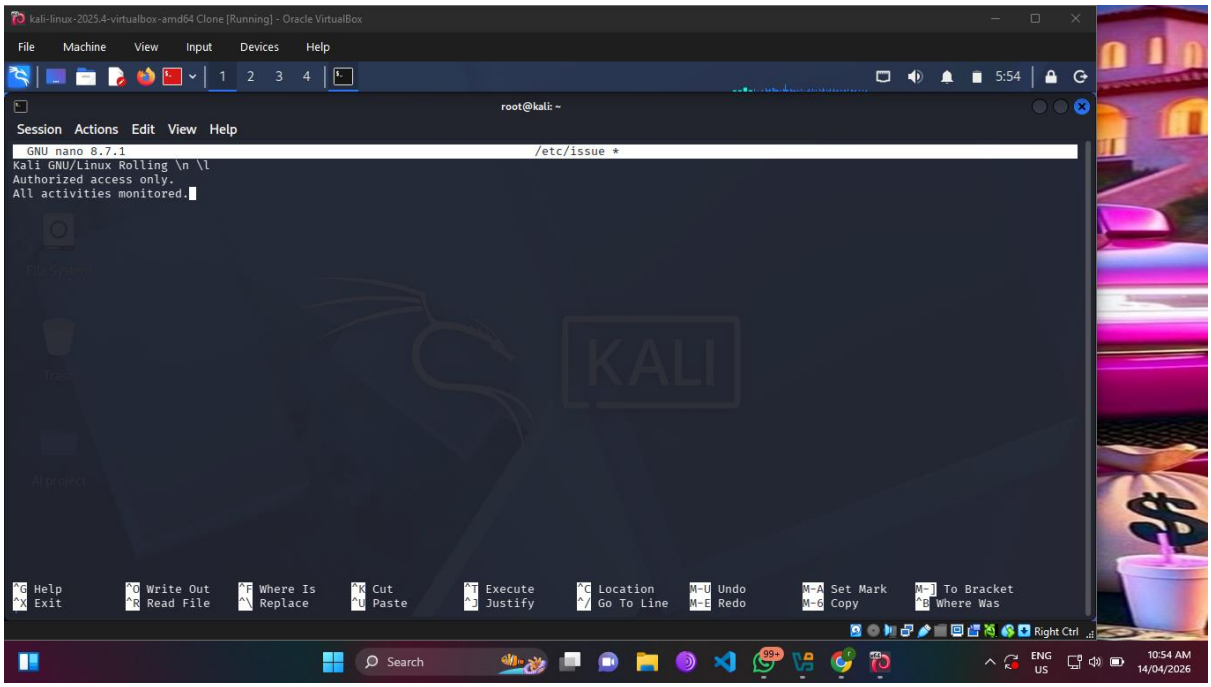

Figure 11: Screenshot of backup file listing and verification

Task 12: Creating a Login Security Banner

The final task required displaying a security warning banner before user login. I used the command echo -e "Authorized access only.\nAll activities monitored." | sudo tee /etc/issue to write the banner to the /etc/issue file. The -e flag enables interpretation of the newline character, creating two separate lines. The /etc/issue file displays its contents before the local console login prompt. When I viewed the file with cat /etc/issue, the output showed the two-line message exactly as written. To make this banner also appear for SSH logins, I copied the file to /etc/issue.net and added Banner /etc/issue.net to the SSH configuration. This banner serves both a legal purpose by notifying users of monitoring and a security purpose by deterring unauthorized access attempts.

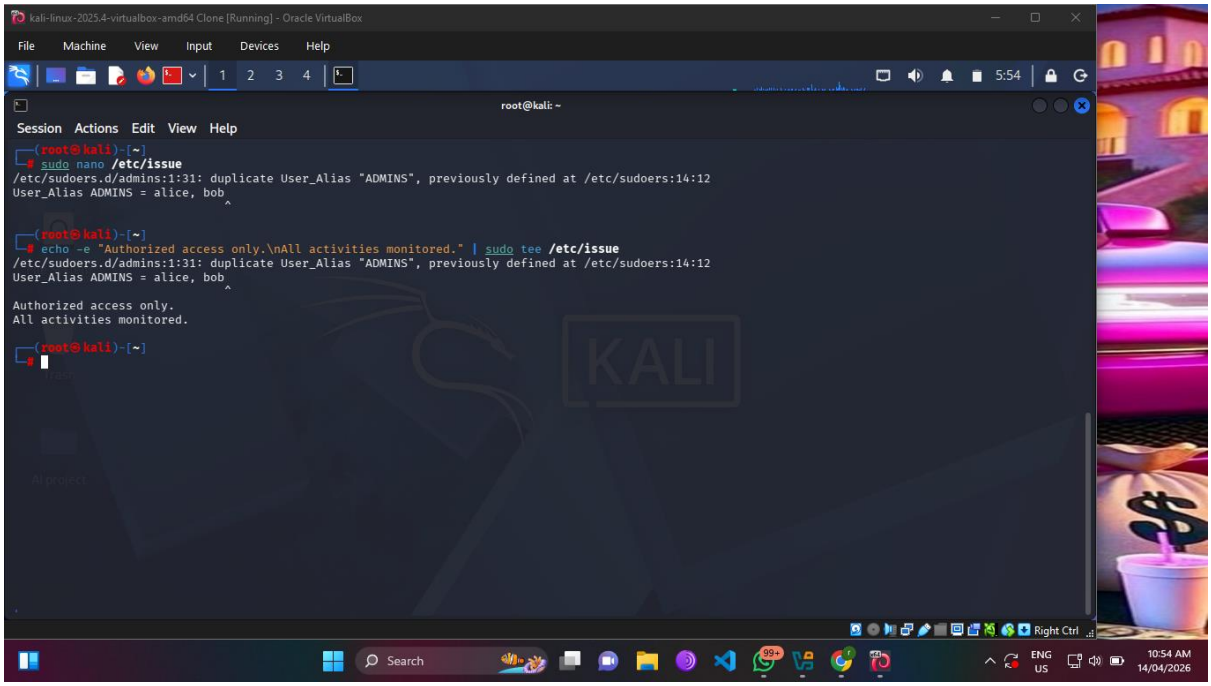

Figure 12 : Screenshot of /etc/issue content

Summary of Lab Outcomes

After completing all twelve tasks, the Linux server has been significantly hardened against common security threats. Administrative access is now restricted to the sysadmins group containing alice and bob, with a 3-minute sudo timeout requiring frequent re-authentication. Remote root login has been disabled, forcing administrators to use their personal accounts and sudo for privilege escalation. Password security has been strengthened through 12-character minimum length requirements including uppercase, lowercase, and numeric characters, with mandatory 90-day rotation and 7-day advance warnings. The system-wide umask of 027 ensures new files are created with secure permissions of 640, preventing unauthorized reading by other users. Account lifecycle management procedures are now in place, demonstrated by the vacation lock of the intern account and the complete offboarding of tempuser including home directory backup. Finally, the login security banner provides legal notice of monitoring and unauthorized access prohibitions. All verification commands produced expected outputs confirming successful implementation of each security control.

Troubleshooting Encountered

During Task 3 when restarting the SSH service, I initially received an error stating Failed to restart sshd.service: Unit sshd.service not found. This occurred because Ubuntu-based systems name the SSH service simply ssh rather than sshd. After identifying the correct service name using sudo systemctl list-units | grep -i ssh, I successfully restarted the service with sudo

systemctl restart ssh. This highlights the importance of understanding distribution-specific differences in system administration.

Conclusion

The lab successfully demonstrated essential Linux system hardening techniques applicable to production server environments. Each task built upon previous configurations, creating layered security that addresses multiple attack vectors including unauthorized access, weak passwords, excessive sudo privileges, and missing legal notifications. The practical hands-on approach using actual commands and verification steps ensures these skills are immediately transferable to real-world system administration scenarios.

References

Garn, D. (2025, April 14). *How to create custom sudo configuration files in /etc/sudoers*. TechTarget. https://www.techtarget.com/searchsecurity/tutorial/How-to-create-custom-sudo-configuration-files-in-etc-sudoers

LabEx. (2025, August 30). *How to secure root access?* https://labex.io/questions/how-to-secure-root-access-941062

LinuC (Linux Professional Institute Certification). (2026, January 8). *OpenSSH サーバーの設定と管理* [OpenSSH server configuration and management]. LPI-Japan. https://linuc.org/study/samples/7876/

man7.org. (2026, January 16). *chage(1) — Linux manual page*. Linux Programmer's Manual. https://www.man7.org/linux/man-pages/man1/chage.1.html

MIT Computer Science & Artificial Intelligence Lab. (2025, February 25). *Linux admin*. Tig.csail.mit.edu. https://tig.csail.mit.edu/operating-systems/csail-linux/linux-admin/

OpenDev / StarlingX Project. (2025, January 31). *Setting a secure umask value (CIS 5.4.2.6)*. In *config-files/umask.sh*. https://opendev.org/starlingx/config-files/src/commit/5df711b1fb9eff45219c1e65ff5f740093d8d741/base-files-config/source/umask.sh

Student Signature: _____

Date: April 14, 2026